

Manual de Treinamento - GitFlow

Equipe-3-ano

Visão Geral

Este manual orienta desenvolvedores sêniores, designers de jogo (front-end), analistas de qualidade e demais membros da equipe no uso do GitFlow para o repositório **Equipe-3-ano**.

Estrutura de Branches

```
main (estável, produção)
├── Sprint01, Sprint02, Sprint03... (releases)
├── Hotfix01, Hotfix02... (correções urgentes)
├── DevelopCSU01, DevelopCSU02... (integração contínua)
│   ├── FeatureBDCSU01 (banco de dados)
│   ├── FeaturePOOCSU01 (programação orientada a objetos)
│   ├── FeatureBACKSU01 (backend)
│   ├── FeatureFRONTCU01 (frontend/UI)
│   └── Ideas5078CSU01 (experimentação júnior)
```

Configuração Inicial

1. Clonando o Repositório

```
# Clone o repositório
git clone https://github.com/ProjetoIntegradorUFV2025/Equipe-3-ano.git

# Entre na pasta
cd Equipe-3-ano

# Configurar usuário (se ainda não configurado)
git config user.name "Seu Nome"
git config user.email "seu.email@empresa.com"

# Verificar branches remotas
git branch -r

# Verifique as branches disponíveis
git branch -a
```

2. Configuração do Git Flow (Opcional)

```
# Instalar git-flow (se disponível)
# Ubuntu/Debian: sudo apt-get install git-flow
# macOS: brew install git-flow

# Inicializar git-flow no projeto
git flow init
```

Papéis e Responsabilidades

Desenvolvedores Sêniores

Responsabilidades:

- Criar e manter branches: Sprint, Develop, Hotfix, Feature (BD, POO, BACK)
- Realizar merges em develop e sprint, após liberação
- Criar Pull Requests para main
- Supervisionar os júniores em ideias

Branches que Criam/Mantêm:

- Sprint<nºSprint> (ex: Sprint01, Sprint02)
- DevelopCSU<código> (ex: DevelopCSU01, DevelopCSU02)
- Hotfix<número> (ex: Hotfix01, Hotfix02)
- FeatureBD<CSU>, FeaturePOO<CSU>, FeatureBACK<CSU>

Designers de Jogo (Frontend)

Responsabilidades:

- Criar e manter branches FeatureFRONT
- Desenvolver interfaces
- Colaborar com sêniores em integrações

Branches que Criam/Mantêm:

- FeatureFRONTCSU<código> (ex: FeatureFRONTCSU01)

Analistas de Qualidade

Responsabilidades:

- Revisar todos os Pull Requests para main, develop e sprint
- Aprovar merges após validação - **ÚNICO papel que pode fazer merge na main**
- Garantir padrões de qualidade

Permissões Especiais:

- Merge em main (após aprovação de PR)
- Aprovação obrigatória para PRs críticos

Gerente de Configuração e Mudança

Responsabilidades:

- Criar tags após merge na main
- Gerar releases no GitHub
- Gerenciar versionamento

Fluxo por Sprint

Início da Sprint N

1. Criação das Branches Base (Sênior)

```
# Atualizar main
git checkout main
git pull origin main

# Criar branch Sprint da nova sprint
git checkout -b Sprint0N
git push origin Sprint0N

# Criar branch Develop para os casos de uso da sprint
git checkout Sprint0N
git checkout -b DevelopCSU0N
git push origin DevelopCSU0N
```

2. Criação de Features (Sênior/Designer)

Para Backend/BD/POO (Sênior):

```
# A partir da develop correspondente
git checkout DevelopCSU0N
git pull origin DevelopCSU0N

# Criar feature específica
git checkout -b FeatureBDCSU0N
# ou
git checkout -b FeaturePOOCSU0N
# ou
git checkout -b FeatureBACKCSU0N

git push origin FeatureBDCSU0N
```

Para Frontend (Designer):

```
git checkout DevelopCSU0N
git pull origin DevelopCSU0N

git checkout -b FeatureFRONTCSU0N
git push origin FeatureFRONTCSU0N
```

Durante o Desenvolvimento

3. Trabalho nas Features

```
# Sempre atualizar antes de trabalhar
git checkout FeatureBDCSU0N
git pull origin FeatureBDCSU0N

# Fazer alterações...
git add .
git commit -m "feat(BD): implementar tabela usuarios para CSU0N"

# Subir alterações
git push origin FeatureBDCSU0N
```

4. Merge Feature → Develop (Sênior)

```
# Finalizar feature
git checkout DevelopCSU0N
git pull origin DevelopCSU0N

git merge FeatureBDCSU0N
git push origin DevelopCSU0N

# Opcional: deletar branch Local da feature
git branch -d FeatureBDCSU0N
```

Final da Sprint

5. Integração Develop → Sprint (Sênior) -> após validação

```
git checkout Sprint0N
git pull origin Sprint0N

git merge DevelopCSU0N
git push origin Sprint0N
```

6. Pull Request Sprint → Main (Sênior cria, Analista aprova)

Criar PR (Sênior):

1. No GitHub, ir para repositório Equipe-3-ano
2. Clicar em “New Pull Request”
3. Base: main ← Compare: Sprint0N
4. Título: Release Sprint 0N - CSU0N
5. Descrição detalhada das mudanças
6. Assignar para Analista de Qualidade

Aprovar e Merge (Analista de Qualidade):

1. Revisar código e testes

2. Verificar se atende critérios de qualidade
3. Aprovar e fazer merge
4. **APENAS o Analista pode fazer este merge**

7. Tag e Release (Gerente de Config)

```
# Após merge na main
git checkout main
git pull origin main

# Criar tag
git tag -a v1.0.N -m "Release Sprint 0N"
git push origin v1.0.N
```

No GitHub: criar Release a partir da tag

Comandos Essenciais

Comandos Diários

```
# Ver status atual
git status
git branch

# Atualizar branch atual
git pull origin <branch-atual>

# Commit padrão
git add .
git commit -m "tipo(escopo): descrição"
git push origin <branch-atual>
```

Comandos de Branch

```
# Listar todas as branches
git branch -a

# Trocar de branch
git checkout <nome-branch>

# Criar nova branch
git checkout -b <nome-branch>

# Deletar branch Local
git branch -d <nome-branch>

# Deletar branch remota
git push origin --delete <nome-branch>
```

Tipos de Commit

- feat: nova funcionalidade
- fix: correção de bug
- docs: documentação
- style: formatação
- refactor: refatoração
- test: testes
- chore: tarefas de build/configuração

Cenários Práticos

Cenário 1: Desenvolvimento Feature BD (Sênior)

```
# 1. Criar feature para caso de uso CSU03
git checkout DevelopCSU03
git pull origin DevelopCSU03
git checkout -b FeatureBDCSU03
git push origin FeatureBDCSU03

# 2. Desenvolver (criar tabelas, stored procedures, etc.)
# ... fazer alterações nos arquivos SQL ...

# 3. Commit e push
git add .
git commit -m "feat(BD): criar tabela produtos e relacionamentos CSU03"
git push origin FeatureBDCSU03
```

```
# 4. Finalizar: merge na develop
git checkout DevelopCSU03
git pull origin DevelopCSU03
git merge FeatureBDCSU03
git push origin DevelopCSU03
```

Cenário 2: Desenvolvimento Frontend (Designer)

```
# 1. Criar feature React
git checkout DevelopCSU05
git pull origin DevelopCSU05
git checkout -b FeatureFRONTCSU05
git push origin FeatureFRONTCSU05

# 2. Desenvolver componentes React
# ... criar components, pages, styles ...

# 3. Commit
git add .
git commit -m "feat(FRONT): implementar tela de login CSU05"
git push origin FeatureFRONTCSU05

# 4. Merge para develop
git checkout DevelopCSU05
git pull origin DevelopCSU05
git merge FeatureFRONTCSU05
git push origin DevelopCSU05
```

Cenário 3: Hotfix Urgente (Sênior)

```
# 1. Criar hotfix a partir da main
git checkout main
git pull origin main
git checkout -b Hotfix03
git push origin Hotfix03

# 2. Corrigir problema
# ... fazer correção ...

# 3. Commit
git add .
git commit -m "fix: corrigir erro de validação login"
git push origin Hotfix03

# 4. PR para main (aprovação do Analista obrigatória)
# Criar PR no GitHub: Hotfix03 → main

# 5. Após merge, sincronizar com develop
git checkout DevelopCSU<atual>
git pull origin main
git merge main
git push origin DevelopCSU<atual>
```

Cenário 4: Review de Pull Request (Analista)

No GitHub:

1. Ir para aba "Pull Requests"
2. Abrir PR pendente de revisão
3. Verificar:
 - Código segue padrões
 - Testes passam
 - Documentação atualizada
 - Não há conflitos
4. Adicionar comentários se necessário
5. Aprovar ou solicitar mudanças
6. Se aprovado, fazer merge (apenas Analista pode mergear na main)

Checklist de Verificação

Antes de Fazer Commit

- Código testado localmente
- Mensagem de commit segue padrão
- Não há credenciais ou dados sensíveis
- Código documentado adequadamente

Antes de Criar PR

- Branch atualizada com base mais recente
- Todos os testes passam
- Não há conflitos
- Descrição clara das mudanças
- Assignado para revisor correto

Para Analistas - Antes de Aprovar PR

- Código segue padrões da equipe
- Funcionalidade testada
- Não há vulnerabilidades aparentes
- Documentação atualizada

- Performance adequada

Final da Sprint

- Todas as features mergidas na develop
- Develop mergida na sprint
- PR da sprint para main criado
- Analista aprovou e fez merge
- Gerente criou tag e release
- Branches ideias removidas
- Documentação no ClickUp atualizada



Regras Importantes

NUNCA FAÇA

- Commit direto na `main`
- Merge na `main` sem aprovação do Analista
- Force push em branches compartilhadas
- Alterar histórico de commits públicos

SEMPRE FAÇA

- Pull antes de começar a trabalhar
- Testes antes de fazer PR
- Mensagens de commit descritivas
- Documentar no ClickUp